

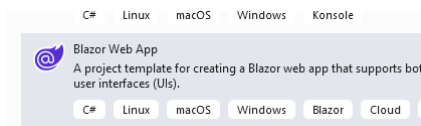
Modul Blazor

Lab: Tortengrafik animiert

Ziel Zusammenspiel Layout, CSS, HTML und Programm Logik

Zeitdauer 15 Minuten.

Erstellen Sie mit Visual Studio eine neue Blazor Web App blazorlab2



Wählen Sie Server Interactive Mode für eine Server basierte Blazor Anwendung.

Blazor-Web-App C# Linux macOS Windows Blazor Cloud Web

Framework ⓘ
.NET 10.0 (Langfristiger Support)

Authentifizierungstyp ⓘ
Keine

☒ Für HTTPS konfigurieren ⓘ

Interaktiver Rendermodus ⓘ
Server

Interaktivitätslocation ⓘ
Global

☒ Beispielseiten einschließen ⓘ

☐ Keine Anweisungender obersten Ebene verwenden ⓘ

☐ Verwenden der .dev.localhost-TLD in der Anwendungs-URL ⓘ

☐ In .NET Aspire-Orchestrierung eintragen ⓘ

Aspire Version ⓘ

Erstellen sie im Verzeichnis Pages die eine neue "Razor Komponente". Fügen Sie in Zeile 1 die Routen Deklaration hinzu

@page „/chartduo“

Fügen sie folgenden HTML Code unterhalb des <h3> Elements ein

Ein SVG Element fügt sich in den HTML Dom Tree ein und lässt beliebige Vector Grafiken darstellen.

```
<button>halb</button>  
<button>drittel</button>
```

```
<svg width="100" height="100" style="transform: rotate(-90deg);border-  
radius:50%;background-color:silver;">  
  <circle r="25" cx="50" cy="50"  
    style=" fill: silver; stroke: blue; stroke-width: 50;  
    stroke-dasharray:60 158;" />  
</svg>
```

Hier wird ein Trick verwendet um ein Kreissegment zu erzeugen. Arcsegements sind in SVG nicht direkt definiert und müssen im normalfall mit aufwändigeren Path Anweisungen erstellt werden.

Compilieren und starten sie das Projekt mit F5. Rufen sie die Url /chartduo auf. Das Ergebnis sieht wie folgt aus.



Nutzen sie die F12 Browser Tools um im SVG Element die Style Anweisung zu entfernen.

```
Elemente  Konsole  Quellen  Netzwerk  Leistung  >>
<button>halb</button>
<!--!-->
<button>drittel</button>
<!--!-->
...
<svg width="100" height="100" style="transform: rotate(-90deg);border-radius:50%;background-color:silver;"> == $0
  <circle r="25" cx="50" cy="50" style=" fill: silver; stroke: blue;
```

Stylen sie die Buttons mit Hilfe des Bootstrap Frameworks in den Browser Tools indem das Class Attribut btn btn -primary hinzugefügt wird.

```
<button class="btn btn-primary">halb</button>
<!--!-->
<button>drittel</button> == $0
```

Beachten Sie die Änderung im Browser.

Da diese Änderung nicht in den Quellcode im Visual Studio Projekt hinzugefügt wird, müssen sie das nun im Visual Studio Editor in der .razor Datei durchführen.

Ergänzen Sie gleichzeitig die Klick Event Handler Methoden

```
<button @onclick="halb" class="btn btn-primary">halb</button>
<button class="btn btn-primary" @onclick="drittel">drittel</button>
```

Für die Berechnung des Kreissegments werden nun zwei einfache Methoden hinzugefügt. Dies im @Code Block

```
public int prozent { get; set; }
const int radius = 25;
void halb()
{
    prozent = Convert.ToInt32(50 * 1.58);
}
void drittel()
{
    prozent = Convert.ToInt32(33 * 1.58);
}
```

Um den Segment Ausschnitt in Grad oder % definieren zu können, muss man die Grundlage dieses CSS Tricks verstehen. Der Radius beeinflusst den benötigte Strichabstand im Strokedasharray.

In diesem Fall $2 \times \pi \times \text{Radius} \sim 158$. Will man 50% Ausschnitt haben $50 \times 158/100$.

Nun wird im Circle Element per Datenbindung der radius und die Segment weite in den HTML Code eingestreut. Ergänzen sie ihren Code wie folgt.

```
<circle r="@radius" cx="50" cy="50"
        style="fill: silver; stroke: blue; stroke-width: 50; stroke-
dasharray: @prozent 158;" />
```

Starten Sie das Projekt und überprüfen Sie das Ergebnis.



Der Wechsel von Zuständen im CSS kann auch animiert werden. Dazu fügen sie in der Style Klasse das Attribut Transition hinzu. Die Position im Style spielt keine Rolle, ob Mitte Ende Anfang solange es in den „“ eingeschlossen wird.

```
<circle r="25" cx="50" cy="50"
        style="fill: silver; stroke: blue; stroke-width: 50;
transition: stroke-dasharray .5s ease; stroke-dasharray: @prozent 158;" />
```

Starten Sie das Projekt.